Frankfurt am Main
Frankfurt University of Applied Sciences

Alternative 7 - „Maze-Spiel“

Entwicklung und Implementierung eines Labyrinths mit Einzelspielermodus in der Shell

Prof. Dr. Christian Baun

Vorgelegt von:
Inna Kuzmina
Maksim Lukianov
Victoria Gordeeva

Studienfach: International/Engineering Business Information Systems - Wirtschaftsinformatik (B.Sc.).

Sommersemester 2021

Frankfurt am Main, 25.06.2021

Inhaltsverzeichnis

1. AUFGABENBESCHREIBUNG 1

1.1 BEDINGUNGEN & ANFORDERUNGEN 2
1.2 ANNAHMEN 2

2. ANALYSE 2

2.1 PROGRAMMIERSPRACHE PYTHON 2
2.2 ALGORITHMUS FÜR DIE LABYRINTH-GENERATION 2
2.3 ALGORITHMUS FÜR DIE INTERAKTIVE NAVIGATION 3
2.4 UML-KLASSENDIAGRAMM 3

3. REALISIERUNG & IMPLEMENTIERUNG 3

3.1 KLASSEN UND METHODEN FÜR DIE LABYRINTH-GENERATION 3
3.2 KLASSEN UND METHODEN FÜR DIE INTERAKTIVE NAVIGATION 6
3.3 GEMEINSAME ANWENDUNG (3.1 & 3.2) 7

4. PROGRAMMBEISPIEL 8

4.1 SCREENSHOTS & ZUSAMMENGEFASSTE BESCHREIBUNG 8

5. ZUSAMMENFASSUNG 10

5.1 SCHLUSSFOLGERUN 10

6. LITERATUR 10

1. Aufgabenbeschreibung

1.1 Bedingungen & Anforderungen

Die Aufgabenstellung setzt voraus, dass ein zufälliges Labyrinth, das über eine Benutzereingabe definierten Größe (2-25) erstellt werden muss, in dem der Benutzer sich interaktiv bewegen könnte. Die Ausgabe ist auf der Kommandozeile zu erfolgen. Es ist nicht möglich, durch Wände hindurchzulaufen. Nur Schritte auf freie Felder sind zulässig. Hat die Spielfigur das Ziel erreicht, gibt das Programm eine entsprechende Nachricht auf dem Bildschirm aus. Es soll auch ausgegeben werden, wie lange der Spieler gebraucht hat, um das Ziel zu erreichen.

1.2 Annahmen

Annahmen für das Programm:

- → Es gibt genau einen Eingang in der oberen linken Ecke und genau einen Ausgang in der unteren rechten Ecke des Labyrinths.
- → Der Irrgarten ist quadratisch.
- → Die Navigation in dem Labyrinth erfolgt über die Pfeiltasten oder A-D-W-S-Tasten.
- → Benutzer hat die Möglichkeit, zwischen den drei von der Größe abhängigen Niveaus auszuwählen oder selbst die Seitengröße zu bestimmen.
- → Sämtliche Versuche, entsprechende Niveaus, Datum, Zeit und verbrachte Zeit werden gespeichert und auf Wunsch auf der Kommandozeile ausgegeben, sodass man seinen Fortschritt verfolgen kann.
- → Benutzer hat die Möglichkeit, die Strecke zum Ausgang angezeigt zu sehen, wodurch er die Taste „h“ drückt.

2. Analyse

2.1 Programmiersprache Python

Zunächst wurde überlegt, ob die als zu verwendende Programmiersprache Python, C oder Shell in Frage kommt. Da alle diese Programmiersprachen für uns unbekannt waren, wurde beschlossen, Python wegen seiner Benutzerfreundlichkeit, einfacher Ablesbarkeit und Syntax auszuwählen. Darüber hinaus ist diese Sprache universell und kann auf mehreren Systemen ausgeführt werden.

2.2 Algorithmus für die Labyrinth-Generation

Es wurde die Entscheidung getroffen, *Backtracking-Algorithmus* zu verwenden. Dieses Algorithmus erstellt ein neues Labyrinth aus einem Gitter von Zellen. Eine zufällige Startzelle wird gewählt, und als besucht markiert. Solange es noch nicht besuchten Zellen gibt, sind dann die folgenden Schritte zu wiederholen:

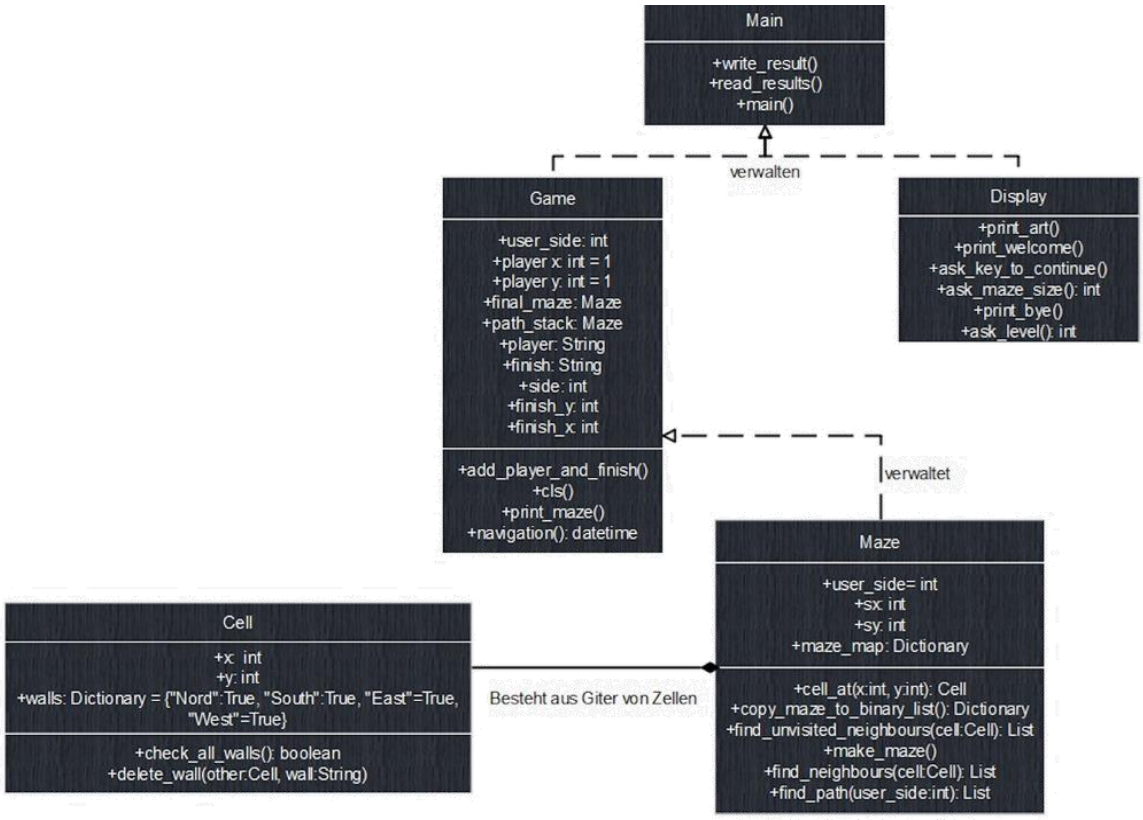
- → Wenn alle Nachbarn besucht wurden, wird auf die letzte Zelle mit den nicht besuchten Nachbarn zurückgesprungen;
- → Einen der nicht besuchten Nachbarn wählen;
- → Die Wand zwischen der aktuellen Zelle und der ausgewählten Zelle löschen;
- → Diese ausgewählte Zelle als besucht markieren;
- → Schließlich wird die ursprüngliche Zelle wieder erreicht und der Vorgang ist abgeschlossen;

Dies ist nicht der effizienteste Algorithmus zum Erzeugen eines Labyrinths, führt er jedoch zu den Labyrinthen mit längeren Pfaden, die attraktiver und schwieriger zu lösen sind. Sie sind voll verbunden, d.h. man von jeder Zelle zu jeder anderen wandeln kann. Zumal ist es schnell, einfach zu implementieren und erzeugt Labyrinthe, die ästhetisch angenehm sind. Von allen Algorithmen erzeugt er am wenigsten Sackgassen und hat dadurch besonders lange und verwinkelte Passagen.

2.3 Algorithmus für die interaktive Navigation

Die Nutzung der *getkey*-Bibliothek für die Navigation des Benutzers via Pfeiltasten ist durch leichte Implementierung, Verständlichkeit und Anwendbarkeit auf Linux, Windows, Mac OS abgesehen von unterschiedlichen tatsächlichen Schlüsselcode geprägt.

2.4 UML - Klassendiagramm



3. Realisierung & Implementierung

3.1 Klassen und Methoden für die Labyrinth – Generation

Zum Aufbau des Labyrinths stehen die *Cell*- und *Maze*-Klassen zur Verfügung.

Die *Cell*-Klasse ist eine Basisklasse. Jede Zelle hat die eindeutigen *x-, y-Koordinaten*, stellt einen Punkt im Gitter dar und ist von den Wänden umgeben. So übernimmt das Attribut x die jeweilige *x-Koordinate*, Attribut *y - y-Koordinate.*

Eine Wand dient zur Zelltrennung in der jeweiligen Richtung Nord-Süd oder West-Ost. Zur Implementierung wird *Dictionary wall_pairs {}* als eine Liste von Schlüssel-Wert-Paaren (die Richtung und der entsprechende Gegensatz) eingesetzt. Eine Zelle besitzt zunächst alle vier Wände, was mithilfe eines Dictionary im Attribut *walls {direction: True...}* implementiert wird.

Die Funktion *check_all_walls()* erfüllt den Zweck der Überprüfung des Vorhandenseins von Wänden für eine gegebene Zelle. Falls alle Wände dabei sind, wird logischer True ausgegeben, sonst False.

```
29      def check_all_walls(self):
30          # Check this cell still have all walls.
31          # All() returns True if all items in the iterable are true, otherwise it returns False.
32          return all(self.walls.values())
```

Die Funktion *delete_wall()* dient dazu, eine Wand zwischen zwei Zellen zu entfernen. Als Übergabeparameters werden die Koordinaten einer anliegenden Zelle und die Wand zum Zerstören angegeben, sodass die Wand sowie einer ursprünglichen Zelle, als auch einer anderen, in einer bestimmten Richtung liegenden, Zelle gelöscht wird.

```
34      def delete_wall(self, other, wall):
35          # Delete the wall between cells - self and other.
36          self.walls[wall] = False
37          other.walls[Cell.walls_pairs[wall]] = False
```

Die *Maze*-Klasse wird als Gitter von der bestimmten Anzahl der Zellen repräsentiert. Die Generation fängt bei der Zelle mit *sx-, sy-Koordinaten* an, weswegen wurden als Attribute *Labyrinthgröße*, *sx*, *sy* und *List maze_map [[]]* ausgewählt.

```
52      self.maze_map = [[Cell(x, y) for x in range(user_side)] for y in range(user_side)]
```

Die Funktion *cell_at() (Zeilen 51-53)* gibt ein Cell- Objekt bei der bestimmten *Koordinaten (y, x)* aus.

Die Funktion *copy_maze_to_binary_list() (Zeilen 55-80)* überschreibt das Labyrinth in ein zweidimensionales Array, das aus 0 und 1 besteht. 1 entspricht einer Wand und 0 - einem freien Feld. Die Seitengröße ergibt sich durch die folgende Gleichung: *side = self.user_side * 2 + 1*, die durch die weitere Verwendung des zweidimensionalen Arrays *maze_in_list [[]]* geeignet ist. Zuerst sind alle Werte gleich 1 (*Zeile 63*), dann werden die Zellen mit einem Schritt von zwei durch 0 ersetzt (*Zeilen 66-68*). Schließlich werden die Ost- und Südwände überprüft, wodurch ein Pfad erstellt und mit 0 markiert wird (*Zeilen 72-78*).

Die Funktion *find_unvisited_neighbours() (Zeilen 82-106)* zeigt die Liste aller nicht besuchten Nachbarn einer bestimmten Zelle an, die vier Varianten von Nachbarn und ihre entsprechenden Indizes aus *(0,0)* hat. Wenn die Koordinaten des Nachbarn nicht über die Labyrinth-Grenze hinausgehen, wird mithilfe von Funktion der *Cell*-Klasse *check_all_walls()* überprüft, ob alle Wände der Zelle vorhanden sind. Wenn diese Bedingung erfüllt ist, wird die aktuelle Zelle in die Liste der nicht besuchten Nachbarn hinzugefügt (*Zeilen 97-108*).

```
97      for direction, (dx, dy) in steps:
98          x2 = cell.x + dx
99          y2 = cell.y + dy
100
101         # Check the neighbour places in the maze
102         if (0 <= x2 < self.user_side) and (0 <= y2 < self.user_side):
103             neighbour = self.cell_at(x2, y2)
104
105             # check_all_walls returns True for only unvisited cells. They will be saved in list.
106             if neighbour.check_all_walls():
107                 neighbours.append((direction, neighbour))
108
109     return neighbours
```

Die Funktion make_maze() veranschaulicht die unmittelbare Verwendung des Backtracking-Algorithmus. Dazu wird eine *cell_stack-Liste* erstellt und wird auch die Startposition der

Generation bestimmt. Solange die Anzahl der besuchten Zellen die Gesamtzahl der Zellen im Labyrinth nicht überschreitet, werden die folgenden Schritte ausgeführt:

→ Suche nach allen nicht besuchten Nachbarn der aktuellen Zelle (*Zeile 126*);

Wenn keine gefunden werden, wird die aktuelle Zelle aus dem Stack entfernt und die zuletzt besuchte Zelle genommen. Die *while*-Schleife fängt von vorne an (*if*-Anweisung (*Zeilen 128-132*))

→ Auswahl einer zufälligen anliegenden Zelle und Entfernung der Wand zwischen der aktuellen und den ausgewählten Zellen (*Zeilen 134-136*);

→ Die aktuelle Zelle wird im cell_stack hinzugefügt. (*Zeile 139*);

→ Der Wert der aktuellen Zelle wird ersetzt und die Anzahl der besuchten Zellen wird um Eins erhöht. (*Zeilen 140-141*);

```
125         while visitednum < totalnum:
126             neighbours = self.find_unvisited_neighbours(current_cell)
127
128             if not neighbours:
129                 # We have reached a dead end: backtrack.
130                 # Take the last visited cell as current cell and remowe it from stack.
131                 current_cell = cells_stack.pop()
132                 continue
133
134             # Choose a random neighbouring cell and delete wall.
135             direction, next_cell = random.choice(neighbours)
136             current_cell.delete_wall(next_cell, direction)
137
138             # Add the current cell in stack and move to the next cell.
139             cells_stack.append(current_cell)
140             current_cell = next_cell
141             visitednum += 1
```

Die Methode *find_neighbours()* ist für die Erfassung sämtlicher Nachbarn zuständig, indem alle vier Richtungen geprüft werden *(Zeilen 143-146).* Gibt es keine Wand zwischen den aktuellen und anliegenden Zellen, wird die letzte mit ihren entsprechenden Koordinaten *(x2, y2)* in *neighbours[]* aufgezeichnet *(Zeile 151-158).*

Die Funktion *find_path()* ist für die Erstellung der Strecke verantwortlich. In der Liste *path_stack[]* werden schrittweise alle Koordinaten fixiert (von den Startkoordinaten bis zu den Endkoordinaten), die zum Ausgang führen und dem Benutzer reibungslos durch den Irrgarten gehen helfen. Die Methode berücksichtigt das Entfernen der schon besuchten Zellen aus *neighbours[] (Zeilen 177-179).* Falls die aktuelle Zelle keine Nachbarn mehr hat, dann wird auf die letzte besuchte Zelle zurückgesprungen und zwei entsprechenden Wände zwischen diesen Zellen erstellt *(Zeilen 181-191).* Die While-Schleife wird solange wiederholt, bis die Endkoordinaten erreicht werden. Die gefundenen Koordinaten der richtigen Strecke werden schrittweise in *path_stack[]* hinzugefügt *(Zeile 197).*

```
177                 if neighbour in path_stack:
178                     backtracking_neighbour = neighbour
179                     neighbours.remove(neighbour)
180
181             if not neighbours:
182                 # We have reached a dead end: backtrack.
183                 # Add walls back and close this path.
184                 for wall in current_cell.walls:
185                     if current_cell.walls[wall] == False:
186                         current_cell.walls[wall] = True
187                         # Backtracking_neighbour is the last visited cell, its wall is also added.
188                         backtracking_neighbour.walls[Cell.walls_pairs[wall]] = True
189                 # Take the last visited cell as current cell and remowe it from stack.
190                 current_cell = path_stack.pop()
191                 continue
192
193             # Choose a random neighbouring cell.
194             next_cell = random.choice(neighbours)
195
196             # Add the current cell in stack and move to the next cell.
197             path_stack.append(current_cell)
198             current_cell = next_cell
```

Die Koordinaten der ermittelten Zellen werden durch die Verwaltung der *Game*-Klasse auf der Kommandozeile ausgegeben, wenn der Benutzer die ‘Hilfe-Taste’ „h“ drückt *(Zeile 300)*. Die Strecke wird anhand der “<>” Zeichen angezeigt.

```
246     def print_path(self, x, y):
247         # At first we decode the grid stack to final maze in the console
248         for cell in self.path_stack:
249             self.final_maze[2 * cell.y + 1][2 * cell.x + 1] = "<>"
250
251         # Start position and player
252         self.final_maze[self.player_x][self.player_y] = "()"
253         self.final_maze[y][x] = self.player
254
255         self.print_maze()
```

3.2 Klassen und Methoden für die interaktive Navigation

Die Prioritätsaufgaben der *Game*-Klasse sind bestimmt für:

- → die Initialisierung der Labyrinth-Größe;
- → die Festlegung der Koordinaten (*sx, sy*) zum Anfang der Labyrinth-Generation durch entstehende Verbindung mit der *Maze*-Klasse;
- → den Aufruf der *maze.make_maze()* und *maze.copy_maze_to_binary_list()* für das Erstellen des Labyrinths.
- → die Festlegung der Benutzer- und Finish-Symbole und ihre entsprechenden Koordinaten;
- → die interaktive Navigation und Zeitbestimmung über die *navigation()*-Methode;
- → die Implementierung der *print_maze()*-Methode.

Das vorher generierte, aus *1 = Wände* und *0 = freie Felder* bestehende Labyrinth wird durch *Unicodezeichen ‘\u2593’* und *Leerzeichen* entsprechend auf der Kommandozeile angezeigt.

```
270         for y in range(self.side):
271             print("\n\t", end="")
272             for x in range(self.side):
273                 if self.final_maze[y][x] == 0:
274                     print("  ", end="")
275                 elif self.final_maze[y][x] == 1:
276                     print('\u2593' * 2, end="")
277                 else:
278                     print(self.final_maze[y][x], end="")
279         print()
```

Die Hauptmethode *navigation()* dient zur Implementierung der interaktiven Navigation per Tastatur anhand der Nutzung der Pfeiltasten oder der Cursortasten nach Ermessen des Benutzers *(Zeilen 276-331)*. Die Realisierung erfolgt durch den Import und die Verwendung von *getkey*-Bibliothek, deren Installation gesondert detailliert in einer separaten Datei “*Getkey Installation”* beschrieben wird. Die Funktion *getkey()* ist für die Bestimmung der von Benutzer eingegebenen Taste zuständig. Die standardisierten Optionen wie *keys.LEFT, keys.RIGHT, keys.UP, keys.DOWN* stimmen den Pfeiltasten *links, rechts, aufwärts, abwärts* überein.
Der folgende Screenshot zeigt die Realisation einer Beispieleingabe durch den Quellcode:

```
303             elif userInput == "a" or userInput == keys.LEFT:
304                 if self.final_maze[y][x - 1] != 1:
305                     self.final_maze[y][x] = 0
306                     x -= 2
307                     self.final_maze[y][x] = self.player
308                     self.print_maze()
```

Bei dem Befehl *“a”/keys.LEFT* wird zunächst überprüft, ob es zwischen den beiden Zellen die Wand gibt *(Zeile 304)*, und lediglich dann darf die x-Koordinate auf das freie Feld hindurchlaufen und sich zwei Schritte nach links bewegen *(Zeile 306)*, solange die y-Koordinate sich während der „horizontalen“ (nach rechts, nach links) Eingaben nicht verändert. Die aktuelle Position des Spielers wird nach der Ausführung vom Befehl überschrieben und durch neue Koordinaten ersetzt. Nach jeweiligem userInput, der die Position der aktuellen Zelle ändert, wird das modifizierte Labyrinth sequentiell auf der Konsole ausgegeben.

Analogisch werden die Aufträge: *„d“/keys.RIGHT, „w“/keys.UP, „s“/keys.DOWN* ausgeführt. Die Anweisungen zur interaktiven Navigation werden wiederholt, solange die *while*-Schleife bei der Abfrage nicht abgebrochen wird, indem der Benutzer die Taste “q” zur Beendigung des Spiels drückt (*Zeile 297*) oder die Koordinaten der aktuellen Zelle mit den *Finish-Koordinaten* übereinstimmen (*Zeile 303*). Mit der *break*-Anweisung wird die Schleife sofort verlassen.

3.3 Gemeinsame Anwendung (3.1 & 3.2)

Display- Klasse ist für die graphische Darstellung des Spiels zuständig und beinhaltet Funktionen zur Ausgabe der Willkommen-(*Zeilen 340-373*) und Tschüss-Nachricht (*Zeilen 393-395*), sowie das Menü für die Steuerung der Labyrinth-Größe (Optionen *[1] - [4])*, das Verlassen des Spiels (Option *[0]*) und die Aufzeichnung von der Statistik zu allen abgeschlossenen Spielen (Option *[5]*).

Benutzer hat eine Gelegenheit zwischen den drei standardisierten Größen (*10x10, 15x15, 20x20*) zu entscheiden oder selbst die Größe zu bestimmen, wozu eine Funktion *ask_maze_size()* dient. Wenn man vertippt und anstatt einer Zahl ein Buchstabe oder ein Sonderzeichen eingegeben hat, wird die entsprechende Fehlermeldung mit der Nachricht angezeigt und wird die Möglichkeit gegeben, den Versuch zu wiederholen.

```
381     def ask_maze_size():
382         print("Please enter the size of maze in the range from 2 to 25:")
383         # Input validation. Only integer numbers are allowed.
384         try:
385             cursor.show()
386             user_side = int(input())
387             # Limitation of maze
388             if not 1 < user_side < 26:
389                 raise Exception
390             cursor.hide()
391
392         except Exception:
393
394             print("\n\tValueError!")
395             print("\tValue must be integer number in the range[2;25]\n")
396             user_side = Display.ask_maze_size()
397
398         return user_side
```

Wenn zumindest ein Spiel erfolgreich abgeschlossen wurde, bietet die Option *[5]* die Möglichkeit zur Vorschau von allen sämtlichen Versuchen und zeigt Datum, Zeit, Größe und die verbrachte zur Aufgabenerledigung Zeit auf.
Die Implementierung der Zeitbestimmung erfolgt durch den Import und die Verwendung des Standard-Moduls für Zeit-Daten *datetime*. Die Aufgabe ist, die Zeitdifferenz zwischen der Anfangszeit und Endzeit des Spiels zu fixieren.
Die Anfangszeit wird unmittelbar direkt nach der Generation des gewünschten Labyrinths in der *Main_Class*-Klasse über die *main()*-Methode gemessen. Die Endzeit wird dagegen in der *Game*-Klasse über die *navigation()*-Methode verzeichnet und zurückgegeben.

```
333                 endtime = dt.datetime.now()
334                 print("\n\n\t\tYou have won!")
335                 break
336             return endtime
```

Die Zwei Ausgangssituationen sind vorherzusehen:

I. Der Benutzer beendet nicht das Spiel, indem er die Taste “q“ zum Ausgang drückt, deshalb wird die Zeitdifferenz gleich Null. Das wird mit der if-Anweisung geprüft und eine String-Zeichenkette “The game is not over” gespeichert.

II. Der Benutzer beendet das Spiel, die Zeitdifferenz wird in der *Main_Class*-Klasse über die *main()*-Methode verzeichnet *(Zeile 511-522).*

→ Die Ausgabe der für die Aufgabenerledigung verbrachten Zeit hängt davon ab, ob die Variable *difftime* weniger oder mehr als eine Minute ist.

→ Die Ergebnisse werden über *write_result()*-Methode dokumentiert und über *read_result()* aufgezeichnet *(Zeilen 453-465).*

```
511         difftime = endtime - starttime
512
513         # Show time of game
514         if difftime.total_seconds() == 0.0:
515             strtime = "The game is not over"
516         elif difftime.total_seconds() >= 60.0:
517             strtime = "%.f" % (difftime.total_seconds() // 60) + ' min ' + "%.1f" % (
518                         difftime.total_seconds() % 60) + ' sec'
519         else:
520             strtime = "%.1f" % difftime.total_seconds() + ' sec'
521
522         print("\n\t\tResult:", strtime)
523
524         # Write result to results.txt
525         # d - day, b - month, Y - year, X - local time
526         Main_Class.write_result(
527             endtime.strftime("%d %b %Y %X") + " : " + str(level) + "x" + str(level) + " : " + strtime + "\n")
```

Main_Class-Klasse vereint und verknüpft sämtliche Klassen des Programms, wodurch auf die Attribute, Objekte, Methoden anderer Klassen zugegriffen wird (beispielsweise (Zeilen 479-500)).

Die Funktionen des Moduls Cursor *cursor.hide()* und *cursor.show()* blenden den blinkenden Cursor aus bzw. blenden ihn wieder ein, wodurch sich die Lesbarkeit bei der Programmausführung verbessert.

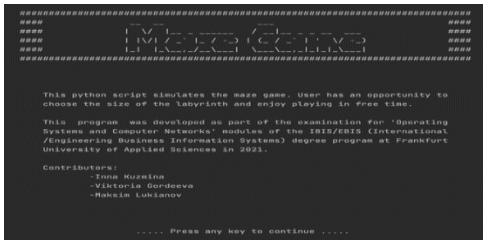
```
479     try:
480         # Menu
481         level = Display.ask_level()
482
483         if level == 0:
484             break
485         elif level == -1:
486             Game.cls()
487             Main_Class.read_results()
488             Display.ask_key_to_continue()
489             continue
490
491         # Create the game
492         game = Game(level)
493         game.add_player_and_finish()
494         game.print_maze()
495
496         starttime = dt.datetime.now()
497         endtime = game.navigation()
498         # Navigation returns time when game is ended.
499
500         difftime = endtime - starttime
```

4. Programmbeispiel

4.1 Screenshots & Zusammengefasste Beschreibung

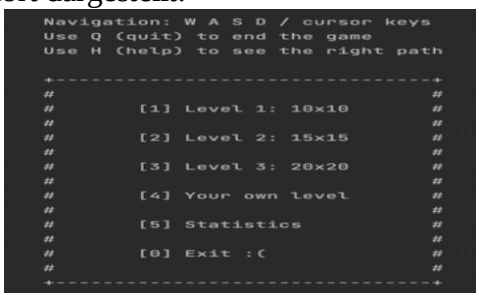
Schritt 1:

Sobald das Kommando ausgeführt ist, erscheint kurze Information zum Spiel auf dem Bildschirm.



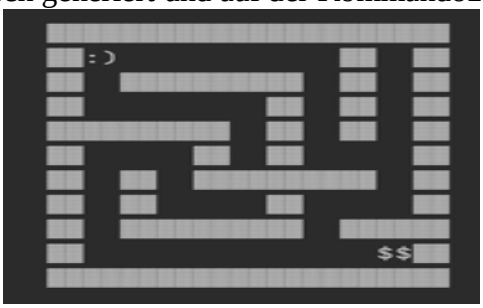
Schritt 2:
Der Benutzer soll eine beliebige Taste auf der Tastatur drücken (z.B. Enter-Taste).

Schritt 3:
Das folgende Menü wird sofort dargestellt.

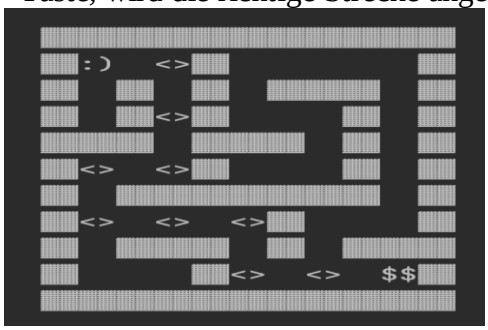


Schritt 4:
Der Benutzer soll eine gewünschte Labyrinth-Größe wählen (z.B. die Option [4] wird gewählt, die Labyrinth-Größe gleich fünf gesetzt).

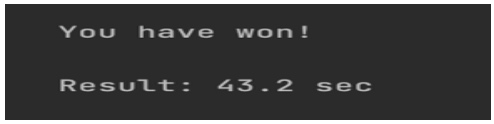
Schritt 5:
Der Irrgarten wird automatisch generiert und auf der Kommandozeile ausgegeben.



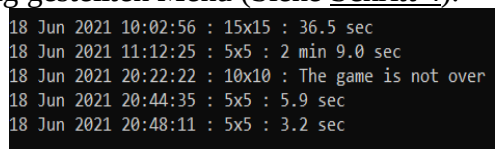
Schritt 6:
Drückt der Benutzer die “h”- Taste, wird die richtige Strecke angezeigt.



Schritt 7:
Der Benutzer soll durch das Labyrinth gehen. Sobald ‘☺’ dem ‘$$’ zusammentrifft, werden “You have won!” – Anweisung und für den Labyrinth-Durchgang verbrachte Zeit ausgegeben. Eine weitere beliebige Handtaste ist wieder zu drücken.



Schritt 8:
Es wird angenommen, dass der Benutzer seine Ergebnisse sehen möchte. Dazu wählt man die Option [5] im zur Verfügung gestellten Menü (Siehe Schritt 4).



Schritt 9:
Zur Beendung ist die Option [0] zu wählen. Es wird die Tschüss-Nachricht ausgegeben.

```
Exiting...

Bye!
```

5. ZUSAMMENFASSUNG

Zusammenfassend wird es festgestellt, dass die zugewiesene Aufgabe erfolgreich erledigt wurde. Die Generation und interaktive Navigation des Labyrinths wurden anhand der Verwendung notwendiger Bibliotheken, nämlich *getkey*, *datetime*, *traceback*, *random*, *cursor* realisiert. Zu dieser Implementierung wurden die oben beschriebenen entsprechenden Klassen und Methoden eingesetzt. Das resultierende Ergebnis lässt dem Spieler eine beliebige Größe innerhalb des Intervalls von 2 bis 25 des Labyrinths zu wählen und sich durch das benutzerfreundliche Menü im Spielraum gut auskennen.

6. LITERATUR

→ Vor- und Nachteilen von Python: https://de.photo-555.com/8888982-what-are-the-benefits-and-limitations-of-using-python#:~:text=Die%20meisten%20Python-Programmierer%20würden%20zustimmen%2C%20dass%20der%20größte,kann%20auch%20den%20Benutzern%20Ihres%20Programms%20zugute%20kommen ;

→ (1) Maze-Generation: Algorithmus: http://weblog.jamisbuck.org/2011/2/7/maze-generation-algorithm-recap ;

→ (2) Maze-Generation: Algorithmus: https://en.wikipedia.org/wiki/Maze_generation_algorithm ;

→ Python-Tutorium: https://docs.python.org/3/tutorial/index.html ;

→ Artikel “Making a Maze”: https://scipython.com/blog/making-a-maze/ ;

→ You-Tube Tutorium: https://www.youtube.com/watch?v=_u_Uv5cpPnc ;

→ Python-Tutorium: YouTube-Kanal «Programmieren Starten», Crashkurs: https://www.youtube.com/watch?v=oxXAb8IikHM&list=PL_pqkvxZ6ho3u8PJAsUU-rOAQ74D0TqZB ;

→ Steyer, Ralph. *Programmierung in Python: Ein kompakter Einstieg für die Praxis*, 1st ed., Wiesbaden: Springer Fachmedien Wiesbaden, 2018;

→ Theis, Thomas. *Einstieg in Python*, 5., aktualisierte Auflage, Bonn: Rheinwerk Verlag, 2017;

→ Kaminski, Steffan. *Python 3*, Berlin, Boston: De Gruyter Oldenbourg, 2016;